

|                                                                                   |                                                                                                   |                                      |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------|
|  | INGENIERÍA EN INFORMÁTICA – PLAN 2014<br>PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE |                                      |
|                                                                                   | APUNTE DE EXPRESSION LANGUAGE                                                                     | VERSIÓN: 1.3<br>VIGENCIA: 13-04-2020 |

## EL (Expression Language)

Expression Language (EL) surge como una forma de acceder a datos dentro de una página JSP sin necesidad de escribir código Java, favoreciendo la construcción de vistas más claras, legibles y mantenibles.

No es un lenguaje de programación en sí mismo, su único propósito es:

- Referenciar objetos y sus propiedades.
- Escribir expresiones simples.

Las expresiones Expression Language se pueden utilizar con tres valores de atributos distintos. En primer lugar, pueden aplicarse cuando un valor de atributo tiene una sola expresión. En segundo lugar, pueden utilizarse cuando el valor del atributo contiene una o más expresiones rodeadas o separadas por texto. Por último, pueden utilizarse si el valor del atributo solo contiene texto.

| Expresiones EL        | Implementación                                                         |
|-----------------------|------------------------------------------------------------------------|
| Una sola expresión    | <code>&lt;xyz.tag value="{expresión}"/&gt;</code>                      |
| Una o más expresiones | <code>&lt;xyz.tag value="texto{expresión}texto{expresión}"/&gt;</code> |
| Solo texto            | <code>&lt;xyz.tag value="texto"/&gt;</code>                            |

Las expresiones se componen de:

- *Identificadores*. Hay once identificadores reservados que corresponden a once objetos implícitos. El resto de los identificadores sirven para crear variables.
- *Literales*. Son números, cadenas delimitadas por comillas simples o dobles, y los valores true, false, y null.
- *Operadores*. Permiten comparar y operar con identificadores y literales.
- *Operadores de acceso*. Se usan para referenciar propiedades de los objetos.

Es importante destacar que Expression Language no está pensado para implementar lógica de negocio, sino para acceder y presentar datos dentro de la vista. Las operaciones que se realizan son simples y orientadas a facilitar la construcción del contenido dinámico.

### Operadores

#### Operadores matemáticos

| Descripción                  | Símbolo | Expresión de ejemplo | Resultado del ejemplo |
|------------------------------|---------|----------------------|-----------------------|
| Multiplicación               | *       | <code>{2*4}</code>   | 8                     |
| División                     | / o div | <code>{5/2}</code>   | 2.5                   |
| Resto de una división entera | % o mod | <code>{5%2}</code>   | 1                     |
| Suma                         | +       | <code>{2+2}</code>   | 4                     |
| Resta                        | -       | <code>{7-2}</code>   | 5                     |

#### Operadores de comparación

|                                                                                                                     |                                                                                                   |                                      |
|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------|
| <br><b>UNIVERSIDAD Blas Pascal</b> | INGENIERÍA EN INFORMÁTICA – PLAN 2014<br>PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE |                                      |
|                                                                                                                     | APUNTE DE EXPRESSION LANGUAGE                                                                     | VERSIÓN: 1.3<br>VIGENCIA: 13-04-2020 |

| Descripción       | Símbolo | Expresión de ejemplo       | Resultado del ejemplo |
|-------------------|---------|----------------------------|-----------------------|
| Igualdad          | == o eq | <code>\${2 == '2'}</code>  | Verdadero             |
| Desigualdad       | != o ne | <code>\${2 != 2}</code>    | Falso                 |
| Menor que         | < o lt  | <code>\${2 &lt; 2}</code>  | Falso                 |
| Mayor que         | > o gt  | <code>\${3 &gt; 2}</code>  | Verdadero             |
| Menor o igual que | <= o le | <code>\${2 &lt;= 2}</code> | Verdadero             |
| Mayor o igual que | >= o ge | <code>\${1 &gt;= 2}</code> | Falso                 |

### Operadores lógicos

| Descripción | Símbolo  | Expresión de ejemplo                             | Resultado del ejemplo |
|-------------|----------|--------------------------------------------------|-----------------------|
| Negación    | ! o not  | <code>\${!(2 == 2)}</code>                       | Falso                 |
| Y           | && o and | <code>\${(2 == 2) &amp;&amp; (2 &gt;= 0)}</code> | Verdadero             |
| Ó           | o or     | <code>\${(2 == 2)    (2 != 2)}</code>            | Verdadero             |

### Operadores de acceso

Para acceder a `object.property` se emplea: `object['property']`. Por ejemplo, `${param['login']}`

### Operador empty

El operador empty comprueba si una colección o cadena es vacía o nula. Por ejemplo, `${empty param.login}`

Otra manera de hacerlo es usando la palabra clave null. Por ejemplo, `${param.login == null}`

### Objetos implícitos

Los objetos implícitos permiten acceder a información disponible en distintos contextos de la aplicación sin necesidad de instanciarlos manualmente. Facilitan el acceso a datos provenientes de la solicitud, la sesión y la aplicación.

| Objeto           | Descripción                                                                                                     |
|------------------|-----------------------------------------------------------------------------------------------------------------|
| pageContext      | Proporciona acceso a información general del entorno, incluyendo request, session y otros objetos del servidor. |
| pageScope        | Contiene valores y nombres de atributos centrados en la página.                                                 |
| requestScope     | Contiene valores y nombres de atributos centrados en la solicitud.                                              |
| sessionScope     | Contiene valores y nombres de atributos centrados en la sesión.                                                 |
| applicationScope | Contiene valores y nombres de atributos centrados en la aplicación.                                             |
| param            | Permite obtener parámetros enviados en el request.                                                              |
| paramValues      | Permite acceder a múltiples valores para un mismo parámetro (por ejemplo, checkboxes).                          |
| header           | Permite acceder a los encabezados de la solicitud HTTP.                                                         |
| headerValues     | Útil cuando un encabezado tiene más de un valor.                                                                |
| cookie           | Permite acceder a las cookies enviadas por el cliente.                                                          |

|                                                                                                                               |                                                                                                   |                                      |
|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------|
| <br><b>UNIVERSIDAD</b><br><b>Blas Pascal</b> | INGENIERÍA EN INFORMÁTICA – PLAN 2014<br>PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE |                                      |
|                                                                                                                               | APUNTE DE EXPRESSION LANGUAGE                                                                     | VERSIÓN: 1.3<br>VIGENCIA: 13-04-2020 |

|           |                                                        |
|-----------|--------------------------------------------------------|
| initParam | Alberga parámetros de inicialización de la página Web. |
|-----------|--------------------------------------------------------|

### Funciones EL

En algunos casos, Expression Language puede ampliarse mediante funciones personalizadas que permiten reutilizar lógica simple definida en clases Java.

Una función EL es mapeada a un método **estático** de una clase Java. El mapeo se especifica dentro de un TLD (Tag Library Descriptor).

Una función en EL puede tomar cualquier número de parámetros. Estos deben ser declarados en el TLD.

Debe respetarse que los nombres de las funciones sean únicos, si no se generará un error al momento de traslación.

Dentro del TLD los parámetros y el tipo de retorno deben ser clases Java totalmente calificadas, por ejemplo, java.lang.String. La sintaxis de la firma de una función EL (<function-signature>) es,

**return\_type static\_function\_name(parameter\_1\_type, ..., parameter\_n\_type)**

Si la firma de la función definida en el TLD no aparece exactamente con la función en la clase Java, un error a tiempo de traslación ocurrirá.

Un TLD es un archivo XML que declara una librería de tags, describiendo la clase que implementa cada tag. Cada TLD puede describir cero o más funciones estáticas. A cada función se le asigna un nombre y método que implementa de la clase Java.

Veamos un ejemplo sencillo,

### Clase Java

```
package ar.edu.ubp.pdc.expressionlanguage;

public class MyStringFunctions {

    public static String lower(String param) {
        return param.toLowerCase();
    }

    public static String upper(String param) {
        return param.toUpperCase();
    }

    public static String wordcap(String param) {
        StringBuilder string = new StringBuilder(param.length());
        String[] words = param.split("\\s");

        for(int i = 0, len = words.length; i < len; i++) {
            string.append(Character.toUpperCase(words[i].charAt(0)))
                .append(words[i].substring(1))
                .append(" ");
        }
        string.trimToSize();
        return string.toString();
    }
}
```

|                                                                                                                         |                                                                                                   |                                      |
|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------|
| <br><b>UNIVERSIDAD<br/>Blas Pascal</b> | INGENIERÍA EN INFORMÁTICA – PLAN 2014<br>PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE |                                      |
|                                                                                                                         | APUNTE DE EXPRESSION LANGUAGE                                                                     | VERSIÓN: 1.3<br>VIGENCIA: 13-04-2020 |

}

### Archivo TLD

```
<?xml version="1.0" encoding="UTF-8"?>
<taglib xmlns="https://jakarta.ee/xml/ns/jakartaee"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-jsptaglibrary_3_0.xsd"
        version="3.0">
    <tlib-version>1.0</tlib-version>
    <short-name>myelfunctions</short-name>
    <function>
        <name>toLower</name>
        <function-class>ar.edu.ubp.pdc.expressionlanguage.MyStringFunctions</function-
class>
        <function-signature>java.lang.String lower(java.lang.String)</function-
signature>
    </function>
    <function>
        <name>toUpper</name>
        <function-class>ar.edu.ubp.pdc.expressionlanguage.MyStringFunctions</function-
class>
        <function-signature>java.lang.String upper(java.lang.String)</function-
signature>
    </function>
    <function>
        <name>toWordcap</name>
        <function-class>ar.edu.ubp.pdc.expressionlanguage.MyStringFunctions</function-
class>
        <function-signature>java.lang.String wordcap(java.lang.String)</function-
signature>
    </function>
</taglib>
```

### Página JSP

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@taglib prefix="fn" uri="/WEB-INF/tlds/myELFunctions.tld"%>

<!DOCTYPE HTML>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>Ejemplo</title>
    </head>
    <body>
        <b>${fn:toLower(";HOLA MUNDO!! :")}</b><br/>
        <b>${fn:toUpper(";hola mundo!! :")}</b><br/>
        <b>${fn:toWordcap(";HOLA MUNDO!! :")}</b>
    </body>
</html>
```

### Resultado

|                                                                                   |                                                                                                   |                                      |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------|
|  | INGENIERÍA EN INFORMÁTICA – PLAN 2014<br>PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE |                                      |
|                                                                                   | APUNTE DE EXPRESSION LANGUAGE                                                                     | VERSIÓN: 1.3<br>VIGENCIA: 13-04-2020 |

**ihola mundo!! :)**  
**¡HOLA MUNDO!! :)**  
**ihola Mundo!! :)**